22AIE205 – Introduction to Python

Lab Assignment 5

Name: AKEPATI SATHISH REDDY

Reg. No : CH.SC.U4AIE25001

Department: AIE (25-29)

Problem Statement 1:

Develop a Python program to implement a BankAccount class for managing a customer's bank account.

The class should contain:

- __init__(self, account_no, holder_name, balance) to initialize account details.
- deposit(amount) to deposit money.
- withdraw(amount) to withdraw money.
- get_balance() to display the current balance.
- __str__() to display the account details in a readable format.
- Use encapsulation by making the balance a private attribute.
- Raise a suitable exception when the withdrawal amount exceeds the available balance.
- Handle invalid input using try-except.
- Demonstrate the program using at least two account objects.

Algorithm :

1. Start the program.
2. Define the BankAccount class.
3. Define the constructor to initialize account number, holder name and balance.
4. Make the balance a private attribute using __balance.
5. Define the deposit() method to add money to the account.
6. Define the withdraw() method to withdraw money from the account.
7. Check whether the withdrawal amount exceeds the available balance.
8. If it exceeds the balance, raise a suitable exception.
9. Otherwise, subtract the withdrawal amount from the private balance.
10. Define get_balance() to display the current balance.
11. Define __str__() to display account details in a readable format.
12. Create at least two BankAccount objects.
13. Perform deposit and withdrawal operations on the accounts.
14. Use try-except to handle invalid input or withdrawal errors.
15. Display the account details and current balances.
16. Stop the program.

Source Code:

200826LabExercise5 > Problem1.py > ...

```python
class BankAccount:
    def __init__(self, account_no, holder_name, balance):
        self.account_no = account_no
        self.holder_name = holder_name
        self.__balance = balance

    def deposit(self, amount):
        self.__balance+=amount

    def withdraw(self, amount):
        if(amount>self.__balance):
            raise Exception("Insufficient balance")
        self.__balance -= amount

    def get_balance(self):

        print("Current Balance: ",self.__balance)
    def __str__(self):
        return "Account No.: "+str(self.account_no)+"\nHolder Name: "+self.holder_name +"\nBalance: "+str(self.__balance)
try:
    account1 = BankAccount(101,"Sathish", 5000)
    account2 = BankAccount(102,"Rakesh", 3000)
    print(account1)
    account1.deposit(1000)
    account1.get_balance()

    account1.withdraw(2000)
    account1.get_balance()
    print()
    print(account2)
    account2.deposit(500)
    account2.get_balance()
    account2.withdraw(1000)
    account2.get_balance()
except Exception as e:
    print("error: ", e)
```

Sample Input(s) and Output(s):

```
PS D:\SEMESTERs\SEMESTER3\22AIE205-Python> python -u "d:
Account No.: 101
Holder Name: Sathish
Balance: 5000
Current Balance:  6000
Current Balance:  4000

Account No.: 102
Holder Name: Rakesh
Balance: 3000
Current Balance:  3500
Current Balance:  2500
```

Explanation and justification of Data Structures:

1. Class: The BankAccount class is used to group account details and operations together. Objects are created for individual bank accounts.
2. Private Attribute: __balance is used to store the account balance privately. This provides encapsulation and prevents direct access to the balance from outside the class.
3. Variable: Account number, holder name and transaction amounts are stored using variables because each account has its own individual values.
4. Exception: An exception is used when the withdrawal amount exceeds the available balance, allowing the invalid transaction to be handled safely.

Problem Statement 2:

Question: Develop a Python program to implement an employee payroll system using inheritance and polymorphism.

Create a base class Employee with:

• Employee ID

• Employee name

• Basic salary

• A method calculate_salary()

Create the following derived classes:

- PermanentEmployee
- ContractEmployee
- Intern

Each derived class should override calculate_salary() and calculate the salary differently according to its employee type.

Requirements:

- Use super() to initialize the common employee attributes.
- Create objects for different employee types.
- Store the objects in a single list.
- Use a loop to call calculate_salary() for every object and display the result.
- Demonstrate how the same method call produces different behaviour for different objects.

Algorithm :

1. Start the program.
2. Define the base class Employee.
3. Define the constructor to initialize employee ID, employee name and basic salary.
4. Define the calculate_salary() method in the base class.
5. Create the PermanentEmployee class by inheriting from Employee.
6. Use super() in its constructor to initialize the common employee attributes.
7. Override calculate_salary() in PermanentEmployee and calculate its salary according to its employee type.
8. Create the ContractEmployee class by inheriting from Employee.
9. Use super() to initialize its common attributes.
10. Override calculate_salary() for the contract employee.
11. Create the Intern class by inheriting from Employee.
12. Use super() to initialize its common attributes.
13. Override calculate_salary() for the intern.
14. Create objects of the different employee types.
15. Store all employee objects in a single list.
16. Use a loop to access each object in the list.
17. Call calculate_salary() for every object using the same method call.
18. Display the employee details and calculated salary.
19. Observe that the same method call produces different results for different employee objects.
20. Stop the program.

Source Code:

200826LabExercise5 > Problem2.py > Employee

```python
class Employee:
    def __init__(self, EmployeeID, EmployeeName, BasicSalary):
        self.EmployeeID = EmployeeID
        self.EmployeeName = EmployeeName
        self.BasicSalary = BasicSalary

    def calculate_salary(self):
        pass

class PermanentEmployee(Employee):

    def __init__(self, EmployeeID, EmployeeName, BasicSalary):
        super().__init__(EmployeeID,EmployeeName, BasicSalary)

    def calculate_salary(self):
        return self.BasicSalary + (self.BasicSalary * 0.2)


class ContractEmployee(Employee):

    def __init__(self, EmployeeID, EmployeeName, BasicSalary):
        super().__init__(EmployeeID,EmployeeName, BasicSalary)

    def calculate_salary(self):
        return self.BasicSalary + (self.BasicSalary * 0.10)

class Intern(Employee):
        def __init__(self, EmployeeID, EmployeeName, BasicSalary):
                    super().__init__(EmployeeID,EmployeeName, BasicSalary)

        def calculate_salary(self):
            return self.BasicSalary

employee = []
employee1 = PermanentEmployee(1001,"Sathish", 5000)
employee2 = ContractEmployee(1002,"Renuka Devi", 40000)
employee3 = Intern(1004,"Suraj", 15000)
```

```
38
39  employee.append(employee1)
40  employee.append(employee2)
41  employee.append(employee3)
42  for em in employee:
43      print("Employee ID:", em.EmployeeID)
44      print("Employee Name:", em.EmployeeName)
45      print("Basic Salary:", em.BasicSalary)
46      print("Calculated Salary:", em.calculate_salary())
47      print()
48
49
```

Sample Input(s) and Output(s):

```
PS D:\SEMESTERs\SEMESTER3\22AIE205-Python> python -u "d:\SEMESTE
Employee ID: 1001
Employee Name: Sathish
Basic Salary: 5000
Calculated Salary: 6000.0

Employee ID: 1002
Employee Name: Renuka Devi
Basic Salary: 40000
Calculated Salary: 44000.0

Employee ID: 1004
Employee Name: Suraj
Basic Salary: 15000
Calculated Salary: 15000
```

Explanation and justification of Data Structures:

- Class: The Employee class stores common employee attributes such as employee ID, name and basic salary. Derived classes reuse these attributes.
- Objects: Separate objects are created for permanent employees, contract employees and interns because each object represents an individual employee.
- List: A list is used to store all employee objects together. This makes it easy to process all employee types using a single loop.

Problem Statement 3:

Develop a Python program to implement a Student class that accepts and processes student marks safely.

The program should:

•Create a Student class with student name, register number and marks.

• Accept marks for 5 subjects from the user.

• Calculate total, average and grade.

• Use try-except to handle:

- Non-numeric marks (ValueError)
- Marks outside the valid range of 0–100 (ValueError or custom exception)
- Any other unexpected input errors.

• Create a custom exception named InvalidMarksError.

• Use raise whenever an invalid mark is entered.

• Use else to display the result when all inputs are valid.

• Use finally to display "Marks processing completed."

• Display the student details using __str__().

Algorithm :

1. Start the program.
2. Define the InvalidMarksError custom exception class.
3. Define the Student class with student name, register number and marks.
4. Define methods to calculate total marks, average marks and grade.
5. Define the __str__() method to display student details.
6. Read the student name and register number from the user.
7. Create an empty list to store marks of five subjects.
8. Read marks for five subjects one by one.
9. Convert each mark into an integer using int().
10. If the mark is less than 0 or greater than 100, raise InvalidMarksError.
11. If a non-numeric value is entered, handle it using ValueError.
12. If an unexpected error occurs, handle it using a general exception.
13. If all marks are valid, create a Student object and calculate total, average and grade using the else block.
14. Display the student details and calculated results.
15. Execute the finally block and display "Marks processing completed."
16. Stop the program.

Source Code:

200826LabExercise5 > Problem3.py > ...

```python
class InvalidMarksError(Exception):
    pass

class Student:
    def __init__(self, name, register_no, marks):
        self.name = name
        self.register_no = register_no
        self.marks = marks

    def calculate_total(self):
        return sum(self.marks)

    def calculate_average(self):
        return self.calculate_total() / len(self.marks)

    def calculate_grade(self):
        average = self.calculate_average()

        if average >= 90:
            return "A"
        elif average >= 80:
            return "B"
        elif average >= 70:
            return "C"
        elif average >= 60:
            return "D"
        else:
            return "F"

    def __str__(self):
        return "Name: " + self.name + "\nRegister Number: " + self.register_no


try:
    name = input("Enter student name: ")
    register_no = input("Enter register number: ")

    marks = []

    for i in range(5):
        mark = int(input("Enter marks for subject " + str(i + 1) + ": "))

        if mark < 0 or mark > 100:
            raise InvalidMarksError("Marks must be between 0 and 100")

        marks.append(mark)

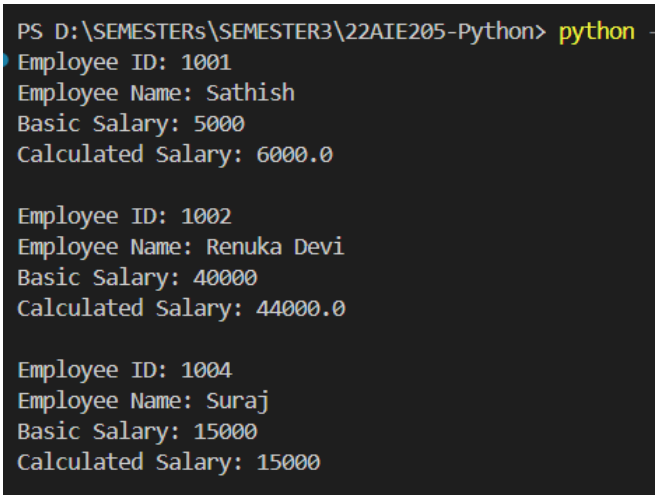
except ValueError:
    print("Invalid input. Please enter numeric marks.")

except InvalidMarksError as e:
    print("Error:", e)

except Exception as e:
    print("Unexpected Error:", e)
else:
    student = Student(name, register_no, marks)
    print("\nStudent Details")
    print(student)
    print("Marks:", marks)
    print("Total:", student.calculate_total())
    print("Average:", student.calculate_average())
    print("Grade:", student.calculate_grade())

finally:
    print("Marks processing completed.")
```

Sample Input(s) and Output(s):



Explanation and justification of Data Structures:

- List: A list is used to store the marks of the five subjects because the number of marks is fixed and the marks need to be stored and processed together. It also allows the use of functions such as sum() and len() to calculate the total and average easily.
- Object: A Student object is used to store the student's name, register number and marks together. This is suitable because the problem is implemented using Object-Oriented Programming.